

Distributed Systems

Lecture3: Classification of Distributed Systems

Mohammad Reza Shakournia

shakournia@ut.ac.ir

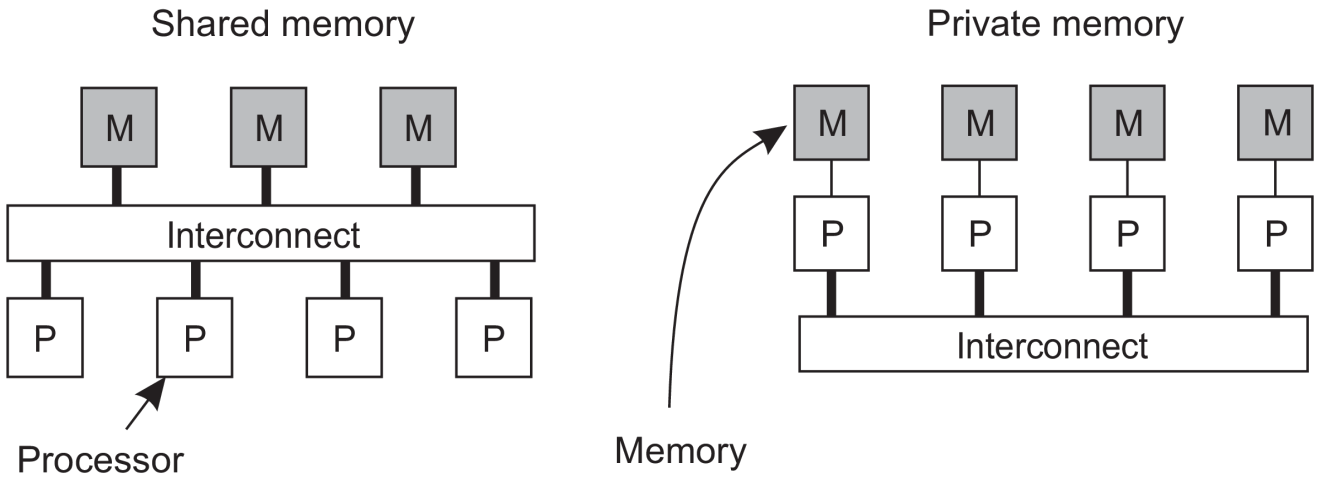
School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Distributed Computing Classification

- Distributed systems can be classified by purpose into :
  - ① high-performance computing,
  - ② general information processing,
  - ③ and pervasive/IoT computing,
- though the boundaries often overlap and hybrids are common.

C1. High-Performance Computing

- High-performance computing more or less started with the introduction of **multiprocessor machines**.
  - Multiple CPUs sharing the same main memory.
- **Multicomputer systems** are separated machines connected by a network with no shared memory.



Multi-Processor does not scale!

- Shared-memory parallelism runs many threads at once over shared data.
  - using synchronization like semaphores.
- **But it doesn't scale well!**
  - Only tens (sometimes hundreds) of CPUs can efficiently share memory, and similar limits appear with multicore processors.
- To scale beyond shared memory, HPC shifted to distributed-memory systems using message passing. (harder and more error-prone)
- Distributed Shared Memory (DSM) that lets a processor **access remote memory** as if local by mapping pages into one virtual address space and fetching remote pages on a page fault (while marking them unavailable at the other node).

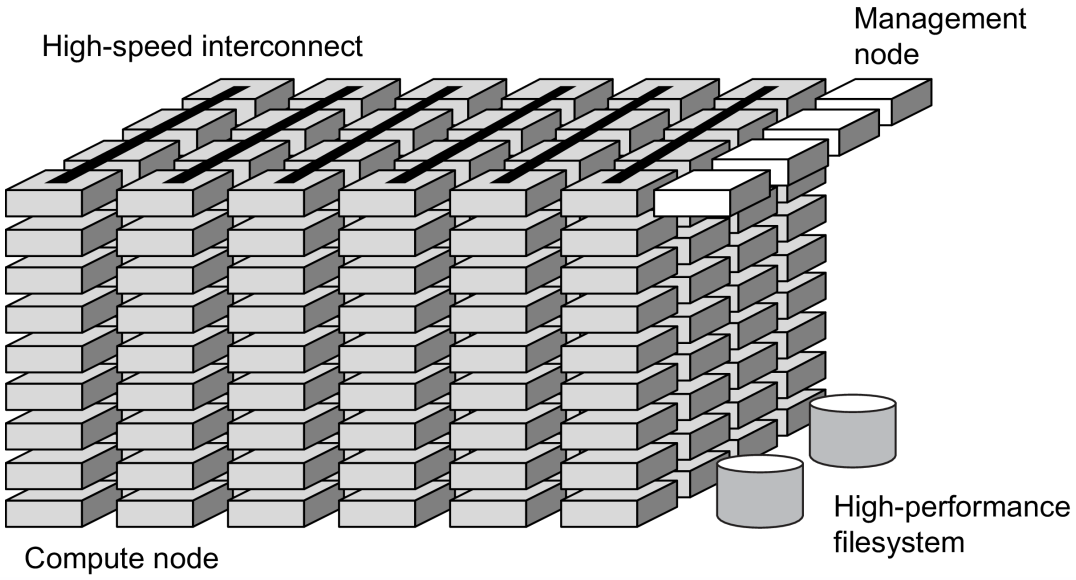
High-Performance Distributed Computing

① **Cluster Computing**: similar nodes, high-speed interconnect, usually the same OS, often with a separate management LAN

② **Grid Computing:** decentralized federations across different administrative domains with heterogeneous hardware/software/network technologies

Cluster Computing

- Cluster computing systems became popular when the price/performance ratio of personal computers and workstations improved.
- Making it feasible to build “supercomputers” from off-the-shelf machines connected by a high-speed network.
- Mainly to run a single program in parallel across multiple nodes.



Cluster Computing

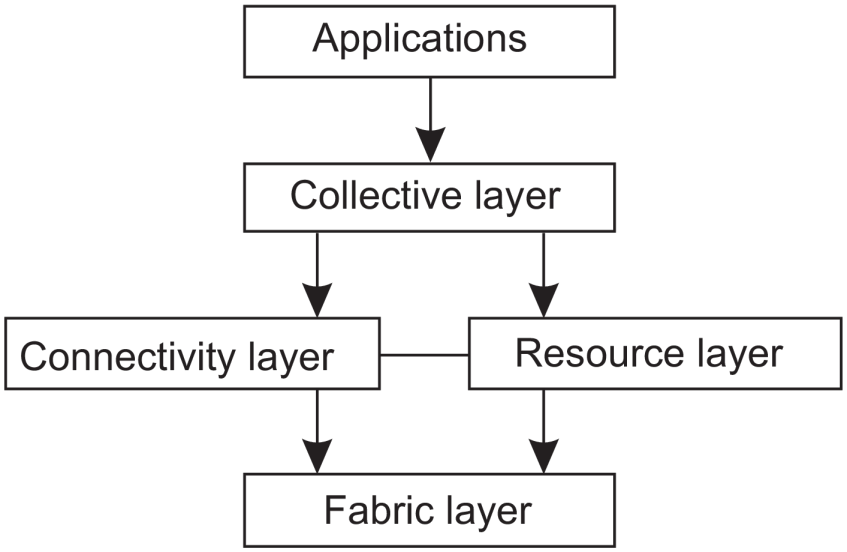
- Modern cluster-based supercomputers have scaled to 100,000+ CPUs (often 8–16 cores each) and use multiple dedicated networks: a high-speed interconnect for computation (often not a single shared network), plus separate management and high-performance storage networks, along with specialized high-speed I/O and remote-access networking.
- Large clusters rely on management nodes to collect user jobs and schedule/distribute tasks across compute nodes (often using multiple management nodes at extreme scale), while compute nodes run a standard OS enhanced with support for communication, storage, and fault tolerance.
- Operating systems in HPC clusters have trended toward lightweight kernels to reduce overhead, but that hurts compatibility/openness; a newer approach is multikernel designs that combine a lightweight kernel with a full OS to support mixed workloads (and features like containers)—most HPC systems run Linux-based stacks, and the performance impact of these newer designs is still being evaluated.

Grid Computing

- Traditional cluster computing is **homogeneous**.
- But we need task-specific node configurations.
- **Grid computing** is inherently heterogeneous, assuming no uniformity in hardware, OS, networks, admin domains, or security policies.
- Grid computing federates resources from multiple organizations into a virtual organization,
  - where participating processes share **access rights** to provided resources.
- Grid computing software mainly focuses on architecture and controlled access to resources across different administrative domains.

Grid Computing Architecture

- Fabric: Access and control of local resources.
- Connectivity: Communication + authentication between sites.
- Resource: Manage one resource and enforce access rights.
- Collective: Coordinate many resources(discover/schedule/replicate).
- Application: User programs running on the grid.



C2. Distributed Information Systems

- In many organizations with many networked applications, interoperability is difficult, so middleware is used to integrate those applications into an enterprise-wide information system.
- At the lowest integration level, clients can bundle multiple server requests into one distributed transaction, ensuring all requests succeed or none are applied.

Definition 1.8. Transaction

A sequence of operations treated as one unit of work—either all of it completes successfully (commit) or none of it takes effect (rollback).

Definition 1.9. Distributed Transaction

A transaction whose operations span multiple networked components (e.g., different servers/databases), coordinated so that all participants commit or all roll back together.

Distributed Transaction Processing

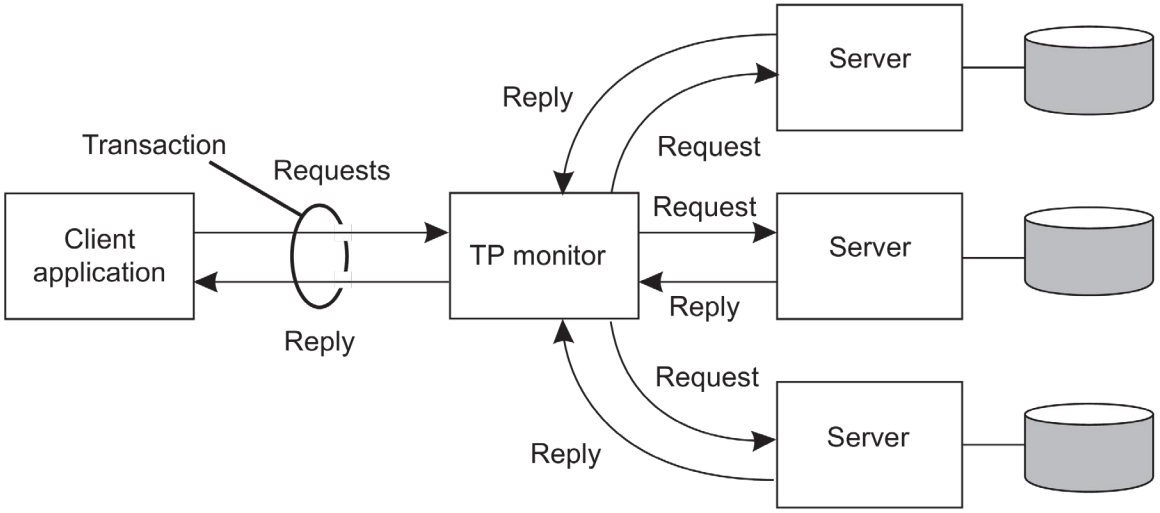
- In database applications, work is performed as transactions.
- Programming with transactions needs **special transaction primitives**.

Primitive	Description
BEGIN_TRANSACTION	Mark the start of a transaction
END_TRANSACTION	Terminate the transaction and try to commit
ABORT_TRANSACTION	Kill the transaction and restore the old values
READ	Read data from a file, a table, or otherwise
WRITE	Write data to a file, a table, or otherwise

- Transactions provide an all-or-nothing guarantee through the ACID properties:
  - **Atomic,**
  - **Consistent,**
  - **Isolated,**
  - **Durable.**

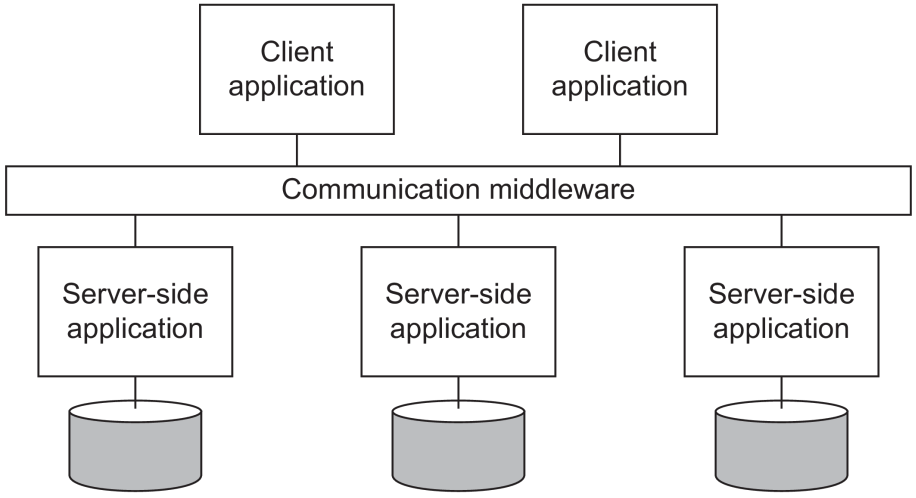
Transaction-Processing Monitor

- A TP monitor is middleware that enables transactions across multiple servers/databases and coordinates commits using a distributed commit protocol.
- It removes the need for applications to coordinate subtransactions themselves by providing this as a reusable middleware service.



Enterprise Application Integration

- As applications became decoupled from the databases, facilities were needed to integrate applications independently of their databases, so application components could communicate directly with each other and not merely by request/reply.
- This need for interapplication communication led to many communication models, so that existing applications could directly exchange information.



Communication Middleware

- **Remote Procedure Calls** (RPC), an application component does a local procedure call, after which the request is packaged as a message and sent to the callee, and the result is sent back and returned as the procedure result.
- **Remote method invocations** (RMI) are essentially the same as RPC, except that an RMI operates on objects instead of functions.
  - RPC and RMI have the disadvantage of tight coupling: caller and callee must both be up and running, and must know exactly how to refer to each other, which led to message-oriented middleware (MOM).
- In **MOM**, applications send messages to logical contact points (often a subject) and subscribe by indicating interest in message types; the middleware delivers messages accordingly—these publish-subscribe systems are an important and expanding class of distributed systems.

C3. Pervasive Systems

- The two previous types of distributed systems we studied were stable distributed systems, because:
  - The nodes (computers) are fixed.
  - Their network connection is usually permanent and high-quality.
  - Using transparency techniques, they make users feel that:
    - failures happen rarely and only sometimes,
    - nodes do not move and stay in place (by hiding their real network location).
- However, matters have changed since the introduction of mobile and embedded computing devices ,and a new class called **pervasive systems** appeared.

Ubiquitous Computing Systems

- It means the system is everywhere and always present. The user interacts with it all the time, and may not even notice the interaction.
- The core requirements for a ubiquitous computing system roughly as follows:
  - **Distribution:** Devices are networked, distributed, and accessible transparently
  - **Interaction:** Interaction between users and devices is highly unobtrusive
  - **Context awareness:** The system is aware of a user's context to optimize interaction
  - **Autonomy:** Devices operate autonomously without human intervention, and are thus highly self-managed
  - **Intelligence:** The system as a whole can handle a wide range of dynamic actions and interactions

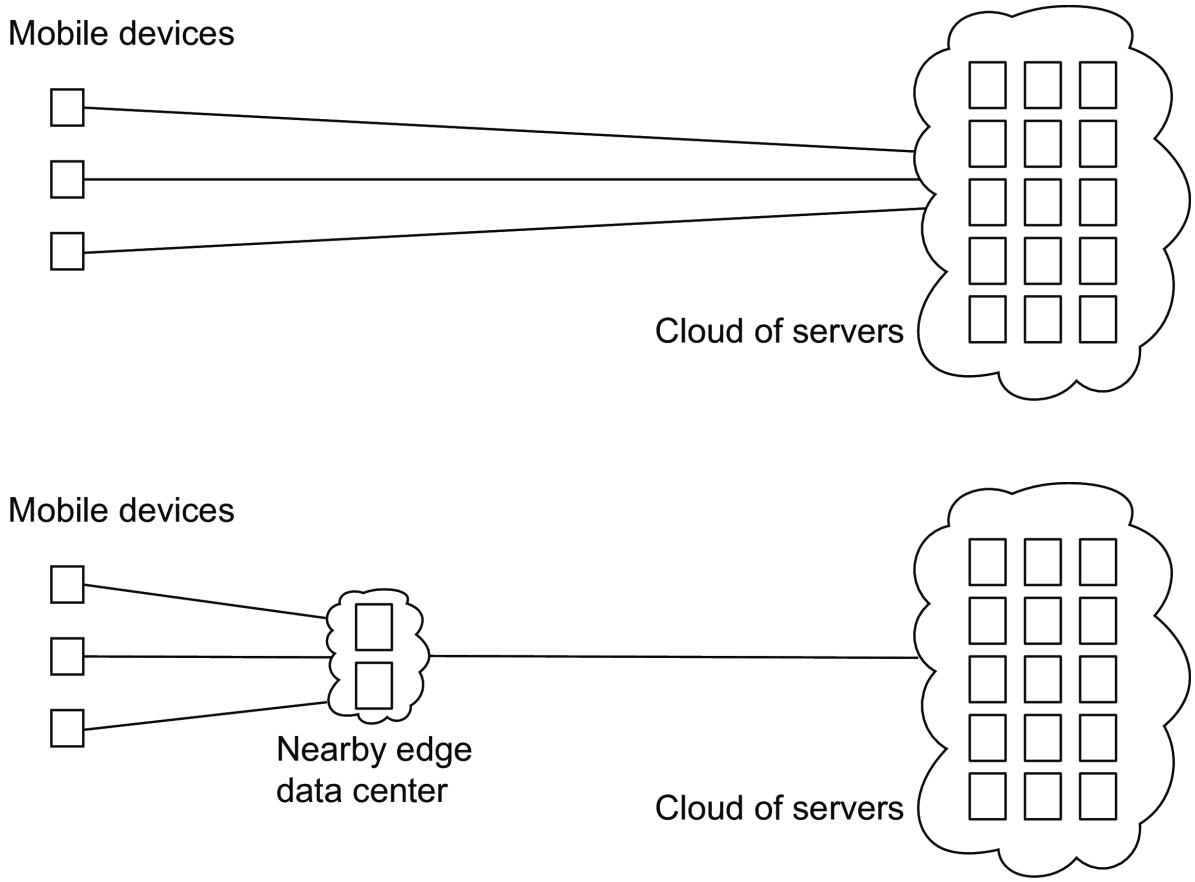
Ubiquitous Computing Systems

- Example of unobtrusive interaction:
  - In a car, the system recognizes that Bob is driving and automatically adjusts the seat and mirrors.
- Context challenge:
  - Turning GPS data into “this street / this shop” and deciding whether this processing happens on the phone or on a server.
- Autonomy examples:
  - Automatic IP address setup (DHCP/Zeroconf), easy device adding (UPnP), and automatic updates.

Mobile Computing Systems

- Mobility often forms an important component of pervasive systems, and many.
- There are several issues that set mobile computing aside to pervasive systems in general.
  - Devices vary widely (e.g., smartphones, tablets, other IP-based devices)
  - Location changes over time
  - Communication is affected by movement
- Mobile computing mostly becomes a single mobile device connected to a stationary server;
  - when the device moves, the connection is handed over by routers, and mobile devices mainly use cloud-based services.

Mobile Computing Systems



Sensor Networks

- A sensor network is a distributed system of many small sensor nodes that collect data from the environment and communicate (often wirelessly) to share and process that data.
- Sensor networks are more than “just sensors”: nodes often collaborate to process sensed data efficiently, so they differ from traditional computer networks.
- They can include tens to thousands of nodes;
  - each node has one or more sensors, and often also actuators (e.g., sprinklers turning on when fire is detected),
  - and because nodes are usually wireless and battery-powered with limited resources, high efficiency and low power use are key design goals.

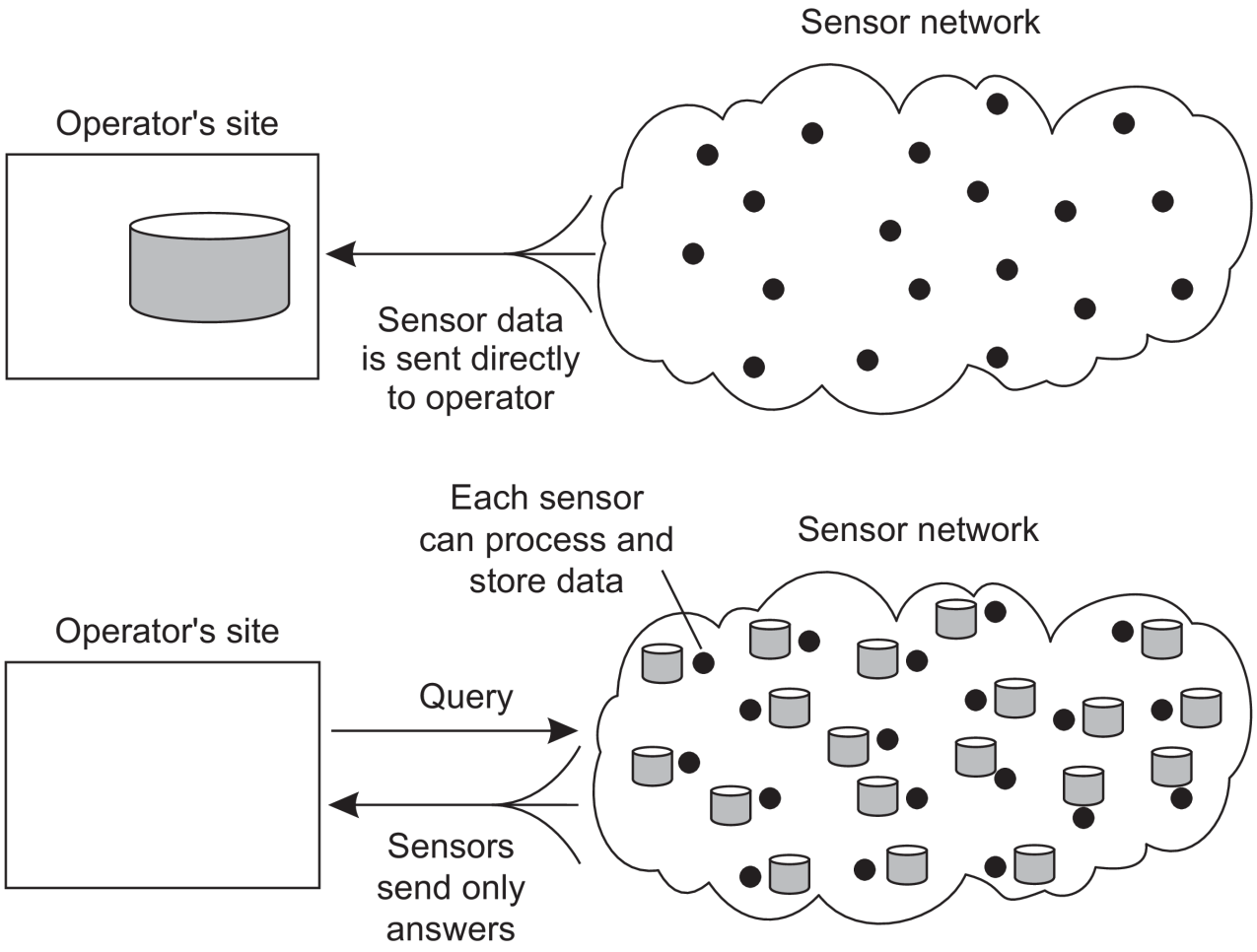
Sensor Networks

- Each sensor node is conceptually like a small computer with an OS-like software layer for:
  - network access, sensor/actuator access, and memory management.
- It usually also provides supporting services :
  - localization, local storage (e.g., flash), and communication facilities like messaging and routing.
- To run sensor applications effectively, extra support is often needed in the form of **middleware** (when nodes are powerful enough to run it).

Sensor Networks

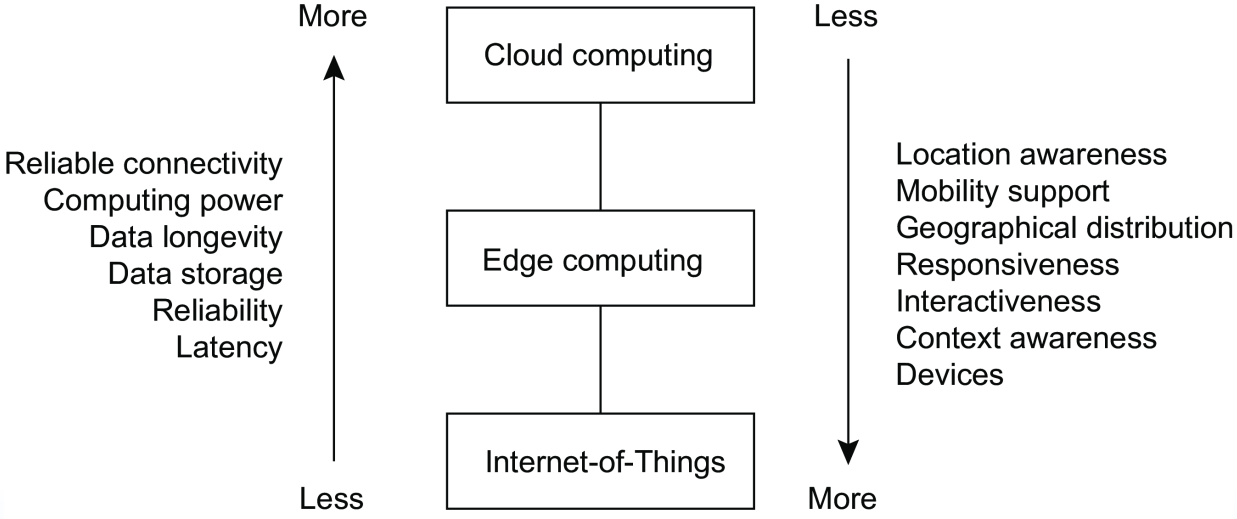
- Data-access model is important in sensor network.
- This can be done by:
  - direct messaging between nodes
  - moving code to where data is
  - using a shared data space view
  - or (very popular) letting the network look like one database for measurement/surveillance queries
- Organizing this as a distributed database has two unattractive extremes:
  - ① sensors send all data to a centralized database (wastes network/energy),
  - ② forward queries to sensors and have the operator aggregate responses (wastes sensor-side aggregation).

Sensor Networks



Cloud, Edge, Things

- With the growth of IoT and cloud services, new architectures have emerged that are usually hierarchical: Device/IoT → Edge → Cloud.
- In this hierarchy, there is a clear trade-off:
  - The higher you go (toward the cloud), the better overall performance, but the number of nodes is smaller.
  - The lower you go (toward IoT/edge), you get lower latency, but there are many more devices and more limitations.



Chapter Wrap-Up

In the next chapter, we will discuss the architectures of distributed systems:

- Architectural Styles,
- Middleware and Distributed Systems,
- Layered-system architectures,
- Symmetrically Distributed System Architectures.